

Documentación Técnica de Drift Detection

22 de marzo de 2026

Índice

1. Drift detection	1
1.1. Objetivo, alcance y relación con el paper	1
1.2. Dependencias y fuentes de datos	2
1.3. Mapa de la interfaz tab a tab	2
1.4. Feature engineering y construcción del target	3
1.5. Preparación de datos: Stage 1 y Stage 2	7
1.6. Selección de variables	7
1.7. Diseño experimental y estrategias	7
1.8. Métricas, tablas y artefactos finales	12
1.9. Validación mediante pruebas	12

1. Drift detection

1.1. Objetivo, alcance y relación con el paper

El módulo `src/drift_detection_app.py` implementa un espacio de trabajo de replicación para el paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. El propio archivo declara que concentra la sección completa de *Drift detection* en una sola fuente Python, y organiza la reproducción de la metodología, la construcción de features, la selección de variables, la ejecución experimental y la verificación de cobertura.

El alcance real del módulo va más allá de un detector puntual de drift. Incluye:

- preparación determinista del dataset de crash prediction para el corredor analizado;
- estrategias batch de recalibración anual;
- estrategias adaptativas con ADWIN, ARF y KSWIN;
- persistencia de checkpoints, cachés de tuning y artefactos SMOTE;
- salidas resumidas alineadas con Appendix A.6–A.9 y Figure 7 del paper.

La configuración de referencia del paper se fija con `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. Además, el corredor de foco se restringe al conjunto fijo de pórticos 11, 12, 14, 15 (`DRIFT_FOCUS_PORTICOS`), mientras que el orden de segmentos usado para la ingeniería de variables del artículo es `15→14, 14→12` y `12→11`.

1.2. Dependencias y fuentes de datos

El módulo asume como fuente principal de flujos la base `Datos/flujos.duckdb` y la tabla `flujos_duckdb`. Para eventos utiliza archivos CSV bajo `Datos/` procesados con `process_accidentes_df`, y para georreferencia y relación entre pórticos utiliza `Datos/Porticos.csv` a través de `load_porticos`, `find_candidate_porticos` y `get_portico_segments`.

La app también depende de librerías opcionales según la etapa:

- `duckdb` para consulta y persistencia de features;
- `optuna` para tuning de hiperparámetros;
- `imblearn` para SMOTE;
- `river` para ADWIN, KSWIN y `AdaptiveRandomForestClassifier`.

1.3. Mapa de la interfaz tab a tab

La función `main()` crea seis tabs con etiquetas reales `Blueprint`, `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments` y `Coverage`. Cada una corresponde a un bloque funcional explícito:

Blueprint `_render_blueprint_tab()` presenta la matriz de replicación del paper, la tabla de trabajo relacionado (`build_related_work_table()`), las secciones/figuras/tablas cubiertas y el plan de migración desde R a Python (`build_python_migration_review_plan()`).

Eventos `_render_events_tab()` carga uno o múltiples archivos de accidentes desde `Datos/`, procesa la asignación a pórticos con `process_accidentes_df()`, conserva los eventos excluidos por falta de mapeo y construye un resumen por tramo para comparar con *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` ofrece tres fuentes: cargar DuckDB existente, recalcular features o reutilizar el dataset en memoria. En modo de cálculo, consulta `flujos.duckdb`, genera variables para pórticos, segmentos y carriles, construye `target`, aplica Stage 1 y Stage 2 y exporta un DuckDB con tablas `raw_features` y `clean_features`.

Feature Selection `_render_feature_selection_tab()` ejecuta filtrado por correlación, ranking con Random Forest y persistencia del resultado de selección dentro del mismo archivo DuckDB de features.

Experiments `_render_experiments_tab()` bloquea el conjunto final de variables, permite seleccionar modelos batch, estrategias, variantes ARF/KSWIN, modos de balanceo y semillas secuenciales, y luego invoca `run_recalibration_experiments()` con soporte de *preview*, *resume* y arranque limpio ignorando checkpoints existentes.

Coverage `_render_coverage_tab()` valida que todas las secciones, figuras y tablas declaradas en `ARTICLE_SECTIONS`, `ARTICLE_FIGURES` y `ARTICLE_TABLES` queden mapeadas a artefactos concretos en Python. Esta verificación se resume con `article_coverage_percentage()`.

1.4. Feature engineering y construcción del target

La ingeniería de variables está concentrada en `_compute_drift_article_features()`, con apoyo de `feature_catalog_table()` y `feature_engineering_reference_table()`. El objetivo es replicar la Table 4 del paper y producir un dataset listo para predicción en grilla de 5 minutos, manteniendo exactamente la semántica operativa del código.

Notación operativa Sea $\Delta = 5$ minutos el ancho del intervalo temporal, y sea

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

la función que asigna un timestamp τ al inicio del intervalo correspondiente. Bajo la configuración artículo por defecto, el corredor usa el orden de pórticos

$$P = (15, 14, 12, 11),$$

las categorías

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

y los segmentos consecutivos

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Cada registro AVI crudo aporta timestamp τ_r , velocidad puntual v_r , categoría c_r , patente π_r , pórtico p_r y carril ℓ_r . La app mapea CATEGORIA a Light/Heavy/Motorcycle con DRIFT_ARTICLE_CATEGORY_MAP, es decir, $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Espacio de variables generado Con el corredor fijo de cuatro pórticos, tres segmentos, tres categorías y tres carriles, la configuración completa genera 260 predictores numéricos antes de agregar target:

- 96 variables por pórtico y tipo de vehículo: 4 métricas base + 4 deltas, para 4 pórticos y 3 categorías;
- 8 variables composicionales por pórtico: `ft_motorcycle_*` y `ft_heavy_*`;
- 72 variables por segmento y categoría: 5 métricas base + 3 deltas, para 3 segmentos y 3 categorías;
- 21 variables globales por segmento: 4 métricas base + 3 deltas, para 3 segmentos;
- 63 variables por carril de origen: 4 métricas base + 3 deltas, para 3 segmentos y 3 carriles.

Si el usuario restringe pórticos o categorías desde la UI, este espacio se reduce, pero las definiciones matemáticas se mantienen invariantes.

Variables por pórtico y tipo de vehículo Para cada intervalo t , pórtico $p \in P$ y categoría $c \in C$, se define el conjunto de observaciones

$$G_{t,p,c} = \{r : b(\tau_r) = t, p_r = p, c_r = c\}.$$

Sobre ese conjunto se calculan las variables con prefijos `flow_`, `vel_`, `sd_`, `den_`:

$$\text{Flow}_{t,p,c} = |G_{t,p,c}|,$$

$$\begin{aligned} \text{Vel}_{t,p,c} &= \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r, \\ \text{Sd}_{t,p,c} &= \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \text{Vel}_{t,p,c})^2}, \\ \text{Den}_{t,p,c} &= \begin{cases} \frac{\text{Flow}_{t,p,c}}{\text{Vel}_{t,p,c}}, & \text{si } \text{Vel}_{t,p,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

En la implementación real, Flow se rellena con 0 cuando no hay observaciones, mientras que Vel, Sd y Den permanecen vacías cuando el grupo no existe o no tiene suficiente soporte. Además, Sd usa la desviación estándar muestral de `pandas`; por tanto, si sólo hay un vehículo en el grupo, la columna queda en NaN.

Los cambios temporales se construyen con primera diferencia dentro de cada serie temporal:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \quad X \in \{\text{Flow}, \text{Vel}, \text{Den}, \text{Sd}\}.$$

Esto produce las columnas `delta_flow_*`, `delta_vel_*`, `delta_den_*` y `delta_sd_*`. Para el primer intervalo donde la variable base está definida, el código fija el delta en 0; si la variable base ya nace vacía, el delta también queda vacío.

Fracciones composicionales por p rtico Sobre el mismo intervalo y p rtico, la app calcula el flujo total

$$\text{Flow}_{t,p}^{\text{tot}} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

Con ello deriva las fracciones

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}_{t,p}^{\text{tot}}}, \quad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}_{t,p}^{\text{tot}}}.$$

Estas son las columnas `ft_motorcycle_*` y `ft_heavy_*`. Si el flujo total del p rtico es cero, ambas fracciones quedan en NaN en vez de forzarse a cero, porque el denominador no representa una mezcla observada.

Matching por segmento entre p rticos consecutivos Las variables segmentales no se calculan a partir de velocidades locales del p rtico, sino de viajes emparejados entre dos p rticos consecutivos. Para cada segmento $s = (p^-, p^+) \in S$, la distancia d_s se obtiene con `_lookup_article_segment_distance_km()`, priorizando metadatos viales y usando fallback geom trico si hace falta.

La ventana m xima de matching depende de la distancia:

$$\tau_s = \max\left(30, \frac{60 d_s}{20}\right) \text{ minutos},$$

porque `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES` = 30 y la velocidad de referencia m nima es 20 km/h. Un par upstream-downstream se acepta si:

- comparte la misma patente π ;
- el paso downstream ocurre despu s del upstream y antes de τ_s ;
- la categor a coincide entre ambos extremos;

- el tiempo de viaje es estrictamente positivo.

La implementación usa `merge_asof(..., direction="backward")` desde el pórtico downstream hacia el upstream y luego elimina duplicados por `plate_clean` y `time_start`, conservando el primer downstream compatible de cada observación upstream. El intervalo del match se ancla al timestamp upstream, es decir, a $b(\tau_{\text{start}})$.

Para cada match m del segmento s , se define el tiempo de viaje

$$\text{TT}_m = \tau_m^{\text{end}} - \tau_m^{\text{start}},$$

y la velocidad espacial

$$\text{SegVel}_m = \begin{cases} \frac{d_s}{\text{TT}_m/1 \text{ hora}}, & \text{si } d_s > 0, \\ \text{NA}, & \text{si no hay distancia válida.} \end{cases}$$

En paralelo, se calcula el indicador de cambio de carril

$$\text{LC}_m = \mathbf{1}\{\ell_m^{\text{start}} \neq \ell_m^{\text{end}}\},$$

si ambos carriles existen; en caso contrario, el match aporta `NaN` a esa componente.

Variables por segmento y tipo de vehículo Sea $M_{t,s,c}$ el conjunto de matches del intervalo t , segmento s y categoría c . A partir de ellos se generan columnas como `flow_light_15_14`, `vel_heavy_14_12` o `cl_motorcycle_12_11`:

$$\begin{aligned} \text{Flow}_{t,s,c} &= |M_{t,s,c}|, \\ \text{Vel}_{t,s,c} &= \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m, \\ \text{Sd}_{t,s,c} &= \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2}, \\ \text{CL}_{t,s,c} &= \frac{1}{|M_{t,s,c}^{LC}|} \sum_{m \in M_{t,s,c}^{LC}} \text{LC}_m, \\ \text{Den}_{t,s,c} &= \begin{cases} \frac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{si } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{en otro caso.} \end{cases} \end{aligned}$$

Aquí $M_{t,s,c}^{LC}$ denota el subconjunto con carriles observados en ambos extremos. Los deltas ΔFlow , ΔVel y ΔDen repiten la misma definición de primera diferencia temporal usada a nivel de pórtico. No existe ΔCL en el código actual.

Variables globales por segmento y por carril de origen El bloque global del segmento agrega todos los matches del intervalo sin separar por categoría:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

Sobre $M_{t,s}$ se generan `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14` y sus deltas temporales. Matemáticamente, las fórmulas son idénticas a las del bloque segmental por categoría, pero reemplazando $M_{t,s,c}$ por $M_{t,s}$.

Para capturar heterogeneidad por carril, el mismo matching se reagrupa por carril de origen:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell_m^{\text{start}} = \ell\}, \quad \ell \in \{1, 2, 3\}.$$

Sobre ese subconjunto se calculan $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$ y sus deltas, produciendo columnas como `vel_lane2_15_14` o `delta_den_lane3_12_11`. Este bloque es importante porque el propio código justifica su inclusión por la relevancia empírica de variables por carril en la réplica de Figure 6.

Reglas de borde y consistencia numérica La implementación distingue entre ausencia de tráfico y ausencia de medida:

- los flujos se rellenan con 0 cuando el grupo no tiene observaciones;
- velocidades, desviaciones, densidades y *lane change fractions* permanecen en NaN si no existe información suficiente;
- los deltas se calculan sobre la versión ya pivotada de cada bloque, por lo que respetan el mismo patrón de faltantes;
- cuando el cálculo se hace por lotes en DuckDB, las features estáticas coinciden con el cálculo monolítico, y los deltas se reinician en los bordes de lote con 0 o NaN; esto está cubierto por `test_drift_batched_flow_features_duckdb_consistency`.

Construcción del target El target se agrega con `_add_interval_accident_target()` después de filtrar accidentes del corredor con `_filter_accidents_for_allowed_porticos()`. Si t_a es el timestamp de un accidente, su marca binaria se asigna al intervalo

$$t^* = b(t_a) - \Delta.$$

Por tanto,

$$y_t = 1 \iff \exists a \text{ tal que } b(t_a) = t + \Delta.$$

Esta definición implementa un *lead time* de un bin: las features de t intentan anticipar accidentes que ocurren en el siguiente intervalo de 5 minutos. La prueba `test_drift_article_features_cover_table4_and_target` valida explícitamente este corrimiento; por ejemplo, un accidente a las 00:06 activa `target = 1` en la fila de las 00:00, no en la de las 00:05.

1.5. Preparación de datos: Stage 1 y Stage 2

La Sección 4.2 del paper se operacionaliza con `apply_missing_data_policy()`, `identify_long_zero_accident_runs()`, `filter_long_zero_accident_runs()` y `run_configurable_preparation_pipeline()`.

- **Stage 1:** elimina variables con más de 1% de valores faltantes y luego elimina intervalos con cualquier predictor faltante en las variables restantes.
- **Stage 2:** elimina ventanas largas sin accidentes, por defecto de al menos 7 días consecutivos con `target = 0`.

La salida incluye un `DataPreparationSummary` con conteo de filas iniciales/finales, variables removidas, ventanas detectadas y filas descartadas. La prueba `test_configurable_preparation_pipeline_respects` valida que ambos stages respondan correctamente a sus selectores.

1.6. Selección de variables

La etapa de selección usa `compute_abs_correlations()`, `drop_highly_correlated_features()` y `rank_features_random_forest()`. Operativamente se ejecuta en dos pasos:

1. remover variables altamente correlacionadas con un umbral configurable;
2. entrenar un Random Forest para ordenar las variables restantes y quedarse con el top- N efectivo para los experimentos.

La misma pestaña genera la réplica de Figure 5 (densidad de $|\rho|$) y Figure 6 (importancias top), y persiste el payload de selección en el DuckDB de features. La prueba `test_feature_selection_payload_persists` cubre esta persistencia.

1.7. Diseño experimental y estrategias

Las estrategias disponibles están definidas por `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf` y `adaptive_kswin`. Los modelos batch soportados por `MODEL_NAMES` son `XGBoost`, `AdaBoost`, `Random Forest` y `NNet`; las variantes online son `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper` y `KSWINseasonal`.

Configuración experimental formal Sea $Y_0 < Y_1 < \dots < Y_J$ la secuencia de años observados en el dataset ordenado por `interval_start`. La app toma como año base Y_0 por defecto, salvo que el usuario fije otro. Para cada intervalo t , el par (x_t, y_t) está formado por las features seleccionadas y el target binario de accidente a un paso. El experimento siempre opera sobre el dataset limpio de Stage 1 y Stage 2.

La orquestación principal en `run_recalibration_experiments()` construye bloques experimentales sobre el producto cartesiano de:

- estrategias;
- modelos batch o variantes online;
- modos de balanceo `none`/`smote` cuando aplica;
- semillas de repetición secuenciales.

Esto significa que el resultado final no es una sola corrida, sino un conjunto de bloques persistidos y reensamblados al final. Formalmente, si B es el conjunto de bloques y R el conjunto de semillas, la cantidad base de ejecuciones es $|B| \times |R|$, a la que se agregan las tareas globales de tuning canónico.

Entrenamiento batch interno y calibración de threshold Las estrategias batch, `adaptive_adwin` y `adaptive_kswin` reutilizan `train_model_with_internal_validation()`. Dado un split de entrenamiento T , el flujo real del código es:

1. dividir T en un split estratificado interno $(T^{\text{fit}}, T^{\text{val}})$ con proporción de validación $\rho = 0,2$ por defecto;
2. sintonizar hiperparámetros sobre T^{fit} con AUC media de validación cruzada estratificada;
3. calibrar el umbral operativo sobre todo T usando scores *out-of-fold*;
4. reentrenar el modelo final con todos los datos de T , y aplicar SMOTE si el bloque lo requiere.

Sea $\Theta_{m,b}$ el espacio de búsqueda del modelo m y del modo de balanceo b . La selección de hiperparámetros resuelve:

$$\hat{\theta} = \arg \max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^K \text{AUC}_k(\theta),$$

donde K es el número efectivo de folds estratificados. Si `balance_mode = smote`, el propio espacio de búsqueda incorpora `sampling_strategy` y `k_neighbors`; si `balance_mode = none`, sólo se buscan hiperparámetros del modelo.

Una vez elegido $\hat{\theta}$, el código genera scores cruzados s_i^{cv} para cada fila $i \in T$ y selecciona el umbral con el criterio de Youden:

$$\hat{\tau} = \arg \max_{\tau} [\text{TPR}(\tau) - \text{FPR}(\tau)] = \arg \max_{\tau} [\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1].$$

Las predicciones duras se definen como

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

Si la calibración *out-of-fold* falla por falta de variabilidad, el código cae a un esquema de respaldo usando el holdout interno T^{val} .

Balanceo con SMOTE El modo `smote` nunca se aplica sobre test o stream, sólo sobre folds de entrenamiento y sobre el refit final. Si la clase minoritaria tiene menos de dos ejemplos, o si la razón minoritaria/mayoritaria ya está por encima del objetivo, el balanceo no se ejecuta. En caso contrario, SMOTE construye un conjunto balanceado $(X^{\text{bal}}, y^{\text{bal}})$ con:

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \text{sampling_strategy},$$

y el vecindario efectivo satisface

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

Los artefactos SMOTE pueden persistirse y reutilizarse por bloque mediante `_load_or_create_smote_artifact()`.

Estrategias batch anuales `run_yearly_strategy()` implementa tres reglas de entrenamiento para cada año objetivo Y_j , $j \geq 1$:

$$\begin{aligned}\mathcal{T}_j^{\text{static}} &= \{t : \text{year}(t) = Y_0\}, \\ \mathcal{T}_j^{\text{period}} &= \{t : \text{year}(t) = Y_j - 1\}, \\ \mathcal{T}_j^{\text{cum}} &= \{t : \text{year}(t) \leq Y_j - 1\},\end{aligned}$$

y el conjunto de evaluación es siempre

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Cada fila de salida contiene `training_year`, `prediction_year`, `model`, `balance_mode`, métricas, umbral, tiempo de entrenamiento y tamaños `n_train/n_test`. En la estrategia `static`, como $\mathcal{T}_j^{\text{static}}$ es idéntico para todo j , el código cachea y reutiliza el bundle entrenado para evitar tuning y refit redundantes; esto está cubierto por `test_run_yearly_strategy_reuses_static_training_bundle`.

El resultado esperado de este bloque es:

- una fila en `yearly_results` por combinación (estrategia, modelo, balance, Y_j , seed);
- una curva ROC segmentada por año objetivo en `roc_payload`;
- tablas Appendix A.6, A.7 y A.8 para `static`, `period_aligned` y `cumulative`, respectivamente.

Recalibración adaptativa con ADWIN `run_adaptive_strategy()` entrena primero un modelo batch sobre el año base Y_0 y luego procesa cronológicamente todo el stream posterior. Para cada fila del stream se calcula un score s_t , una predicción dura $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$ y una señal de error binario

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

La clase `SimpleADWIN` mantiene una ventana W de errores. Para cada corte candidato $W = W_0 \cup W_1$, con tamaños n_0 y n_1 , define:

$$\begin{aligned}\mu_0 &= \frac{1}{n_0} \sum_{e \in W_0} e, & \mu_1 &= \frac{1}{n_1} \sum_{e \in W_1} e, \\ m &= \left(\frac{1}{n_0} + \frac{1}{n_1} \right)^{-1}, & \epsilon &= \sqrt{\frac{1}{2m} \log \left(\max \left(1, 0000001, \frac{4 \log n}{\delta} \right) \right)},\end{aligned}$$

donde $n = |W|$ y $\delta = \text{adwin_delta}$. Se declara drift si

$$|\mu_0 - \mu_1| > \epsilon.$$

Cuando eso ocurre, el código:

- cierra el segmento de predicción acumulado hasta t ;
- registra `drift_date`, `W`, `W0`, `W1` y `remaining_periods`;
- intenta recalibrar el modelo usando las últimas $|W_1|$ observaciones del stream.

El reentrenamiento sólo ocurre si la ventana W_1 tiene al menos $\max(20, \lfloor \text{min_window}/10 \rfloor)$ filas, o el `min_retrain_size` que fije el usuario, y contiene ambas clases. Si no se cumple esa condición, el detector igual corta el segmento, pero el modelo vigente continúa. El resultado esperado es una tabla `adaptive_results` con una fila por drift detectado más un segmento final con `remaining_periods` = 0; estas filas alimentan la parte ADWIN de Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implementa una estrategia genuinamente on-line. Sobre el año base Y_0 , la función `train_arf_with_internal_validation()` separa un bloque final de validación cronológica y usa el bloque inicial para *warm-up* del ensamble ARF. Luego predice secuencialmente sobre el bloque de validación, aprende inmediatamente cada etiqueta y fija $\hat{\tau}$ con el mismo criterio de Youden.

Durante el stream posterior, el modelo se actualiza observación a observación:

$$s_t = f_{t-1}(x_t), \quad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

donde $\mathcal{U}$ representa el paso online `learn_one`. A diferencia de ADWIN y KSWIN, no hay un reentrenamiento batch externo tras un drift: la adaptación está internalizada en el bosque. Por eso el código no duplica esta estrategia por `balance_mode`; siempre usa `not_applicable`.

Los resultados se segmentan por año de predicción, no por drift externo. Cada fila esperada contiene:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;
- métricas y umbral operativo.

En consecuencia, Appendix A.9 muestra a ARF como segmentos anuales online, no como ventanas $W/W_0/W_1$. Las pruebas `test_run_arf_strategy_variants` y `test_recalibration_experiments_support_adapt` verifican esa salida.

Recalibración adaptativa con KSWIN `run_kswin_strategy()` separa claramente el problema de detección de drift covariable del problema de predicción. Primero entrena un modelo batch sobre Y_0 . Luego selecciona las variables monitorizadas con `_select_kswin_monitor_columns()`, que en la implementación actual toma las primeras top-k columnas numéricas del orden de `feature_cols`. No es un ranking online: es una selección determinista de las features ya aprobadas por Feature Selection.

Para cada feature monitorizada f , KSWIN puede operar en dos espacios:

- **KSWINpaper**: usa el valor crudo $x_{t,f}$;
- **KSWINseasonal**: usa un z-score estacional

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

con fallback a perfiles por hora y luego a perfiles globales del año base.

Cada detector usa $\alpha = 10^{-4}$, `window_size` = 300 y `stat_size` = 30, por lo que Appendix A.9 reporta

$$W = 300, \quad W_0 = 270, \quad W_1 = 30$$

como tamaños fijos del detector, no como ventanas aprendidas dinámicamente.

Sea F_t el conjunto de features cuyo detector dispara drift en el instante t . La condición de drift del bloque es:

$$|F_t| \geq \text{vote_threshold}.$$

Cuando eso ocurre, se cierra el segmento de predicción, se registran `vote_count`, `detected_features`, `monitored_features`, y se selecciona una ventana de recalibración temporal:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

donde $D = \text{kswin_retrain_days}$. Si $|\mathcal{R}_t| < \text{kswin_min_retrain_rows}$, el código usa las últimas `min_rows` observaciones; si esa ventana tiene una sola clase, se expande a todo lo observado hasta t . Tras el refit batch, los detectores KSWIN se reconstruyen y se recalientan con la ventana de reentrenamiento.

El resultado esperado de KSWIN es una fila por drift votado más un segmento final, con columnas adicionales como `vote_threshold`, `monitor_feature_count`, `retrain_rows` y `retrain_positive_rows`. Las pruebas `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin` y `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validan precisamente ese comportamiento.

Semillas, tuning global, persistencia y reanudación `run_recalibration_experiments()` no vuelve a tunear desde cero cada bloque batch. Antes de lanzar los bloques, construye tareas de tuning canónico sobre el split base del año Y_0 para cada combinación (modelo, balance) que vaya a ser usada por estrategias batch, ADWIN o KSWIN. Ese tuning global se persiste en `tuning_dir` y luego se inyecta como `fixed_params/fixed_tuning_artifact` en los bloques posteriores. En términos metodológicos, esto impone comparabilidad entre estrategias que comparten familia de modelo, porque todas parten del mismo $\hat{\theta}$ canónico por seed y modo de balanceo.

Cada bloque persiste:

- filas anuales o adaptativas;
- ROC payload segmentado;
- su propio `execution_log`;
- referencias a tuning y a artefactos SMOTE.

El `manifest.json` central mantiene progreso, bloques completados, cachés disponibles y errores. Si una corrida falla a mitad de camino, la siguiente ejecución puede reanudar desde el checkpoint y reutilizar tuning/SMOTE ya calculados; esto está cubierto por `test_preview_recalibration_checkpoint_detects_re` y `test_recalibration_experiments_resume_from_failed_checkpoint`.

Resultados esperados Metodológicamente, la salida completa del experimento debe contener siete productos coherentes entre sí:

- `yearly_results`: una fila por split anual batch, con `strategy`, `prediction_year`, `model`, `balance_mode`, métricas, umbral y tamaños muestrales;
- `adaptive_results`: una fila por segmento adaptativo, ya sea delimitado por drift ADWIN/KSWIN o por año en ARF;
- `summary`: medias por (`strategy`, `model`, `balance_mode`), incluyendo `n_segments` y `n_repetitions`;
- `appendix_tables`: A.6 para `static`, A.7 para `period_aligned`, A.8 para `cumulative` y A.9 para resultados adaptativos;

- `appendix_tables_mean`: las mismas tablas, pero promediadas por semillas secuenciales y enriquecidas con `seed_list` y `run_orders`;
- `average_roc`: curvas ROC medias tipo Figure 7, construidas interpolando cada curva individual sobre una grilla uniforme de 101 puntos en FPR y promediando la TPR:

$$\overline{\text{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\text{TPR}}_{g,j}(u);$$

- `run_manifest` y `execution_log`: trazabilidad reproducible del experimento, incluyendo configuración, seeds, thresholds, tuning, checkpoints, errores y consumo de recursos.

Las pruebas `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_b` y `test_recalibration_experiments_persist_optuna_json` verifican precisamente que ese paquete de resultados exista, sea consistente y pueda persistirse como JSON final.

1.8. Métricas, tablas y artefactos finales

La evaluación usa cuatro métricas base calculadas por `compute_classification_metrics()`: `auc`, `sensitivity`, `specificity` y `error_rate`, además del `threshold` operativo. Sobre esas tablas, el módulo construye:

- resumen consolidado con `summarize_results()`;
- tablas Appendix A.6–A.9 con `format_appendix_tables()`;
- tablas promedio por semillas con `format_appendix_tables_mean()`;
- curva ROC promedio tipo Figure 7 con `build_average_roc_curves()`.

La persistencia principal se divide en tres niveles:

- features exportadas en DuckDB bajo `Resultados/`;
- checkpoints por corrida en `Resultados/drift_recalibration_runs/`, con `manifest.json`, bloques persistidos, tuning cache y artefactos SMOTE;
- JSON final de resultados con `_persist_drift_recalibration_json()`, guardado como `drift_recalibration_o`.

El `run_manifest` registra configuración, semillas, estrategias, tamaños de grilla, features activas y rutas de checkpoint. El `execution_log` almacena trazabilidad de tuning, entrenamiento, errores, reentrenos adaptativos y consumo de recursos.

1.9. Validación mediante pruebas

El archivo `tests/test_drift_detection_app.py` entrega cobertura funcional amplia del pipeline. Entre los casos más relevantes:

- `test_article_coverage_is_100_percent`;
- `test_drift_article_features_cover_table4_and_target_shift`;
- `test_drift_batched_flow_features_duckdb_consistency`;

- `test_end_to_end_recalibration_and_average_roc;`
- `test_recalibration_experiments_compare_balance_modes_end_to_end;`
- `test_recalibration_experiments_persist_optuna_json;`
- `test_recalibration_experiments_resume_from_failed_checkpoint;`
- `test_recalibration_experiments_support_adaptive_arf;`
- `test_recalibration_experiments_support_adaptive_kswin.`

En conjunto, estas pruebas confirman que la documentación de *Drift detection* no describe un diseño aspiracional, sino el comportamiento realmente implementado en el repositorio.